

Experiment 2

Student Name: Sejal

Branch: CSE - AIML

Semester: 5

Subject Name: Soft Computing

UID: 24BAI70050

Section/Group: 24AIT KRG G1

Date : 15/7/26

Subject Code: 24CSH-339

Aim

Build fully connected layer architecture using in-built data sets.

Platform

Anaconda Navigator

Theory

A neural network also known as artificial neural network (ANN) is the basic building block of deep learning. It consists of layers of sigmoid neuron stacked together to form a bigger architecture. Each circle in the above image is a sigmoid neuron. It consists of 3 types of layers, an input layer, an output layer, and hidden layers. All the previous layers are fully connected with the next layer as can be seen in the image so it is sometimes also referred to as the fully connected neural network. Each neuron has its own weight values. The first layer (input) takes the independent variable of data as input. Output layer predicts the class. The number of hidden layers and number of neurons in hidden layers is not fixed and you can choose any number and tinker to get the best results. Neural networks are just the weighted sum of the inputs. So the learning of neural networks is based on updating these weights. We need a method to update the weights. It is based on how good the neural network is performing. Performance of the neural network means how good are the predictions based on the actual labels that are to be predicted. The value at the output layer is calculated by crossing through the neural network and finding the value of each neuron. This process of crossing through the neural network is called forward propagation.

Procedure

1. We feed input data into the neural network.
2. The data flows from layer to layer until we have the output.
3. Once we have the output, we can calculate the error which is a scalar.
4. Finally, we can adjust a given parameter (weight or bias) by subtracting the derivative of the error with respect to the parameter itself.
5. We iterate through that process.

Code

```
from tensorflow.keras import layers, models #layers - to create layers in neural net
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

train_images = train_images.reshape((60000, 28 * 28)).astype('float32') / 255 #image
test_images = test_images.reshape((10000, 28 * 28)).astype('float32') / 255

train_labels = to_categorical(train_labels) # if label 2 then 010000000
test_labels = to_categorical(test_labels)

model = models.Sequential()

model.add(layers.Dense(512, activation='relu', input_shape=(28 * 28,)))
model.add(layers.Dense(256, activation='relu'))
model.add(layers.Dense(128, activation='relu'))
model.add(layers.Dense(10, activation='softmax'))

model.compile(optimizer='adam',
              loss='categorical_crossentropy',
              metrics=['accuracy'])
```

```

model.compile(optimizer='adam',

              loss='categorical_crossentropy',

              metrics=['accuracy'])

history = model.fit(
    train_images,
    train_labels,
    epochs=10,
    batch_size=256,
    validation_split=0.1
)

plt.figure(figsize=(8,5))
plt.plot(history.history['loss'], marker='o', label='Training Loss')
plt.xlabel('Number of Epochs')
plt.ylabel('Loss')
plt.title('Epochs vs Training Loss')
plt.legend()
plt.grid(True)
plt.show()

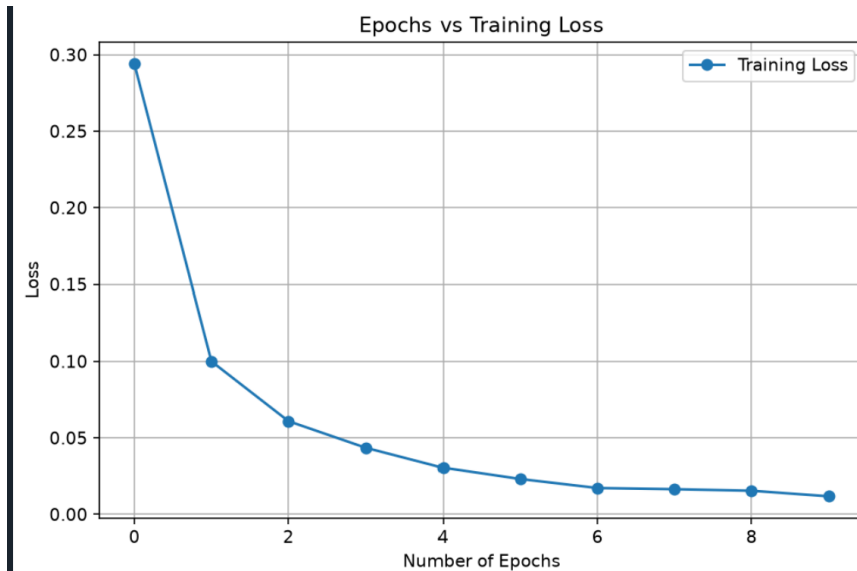
test_loss, test_acc = model.evaluate(test_images, test_labels)

print(f'Test accuracy: {test_acc*100}')
```

Output

```

super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/10
211/211 ----- 3s 9ms/step - accuracy: 0.9160 - loss: 0.2901 - val_accuracy: 0.9683 -
val_loss: 0.1120
Epoch 2/10
211/211 ----- 2s 8ms/step - accuracy: 0.9700 - loss: 0.1005 - val_accuracy: 0.9702 -
val_loss: 0.0916
Epoch 3/10
211/211 ----- 3s 9ms/step - accuracy: 0.9808 - loss: 0.0629 - val_accuracy: 0.9775 -
val_loss: 0.0733
Epoch 4/10
211/211 ----- 2s 9ms/step - accuracy: 0.9861 - loss: 0.0444 - val_accuracy: 0.9798 -
val_loss: 0.0729
Epoch 5/10
211/211 ----- 3s 9ms/step - accuracy: 0.9902 - loss: 0.0299 - val_accuracy: 0.9793 -
val_loss: 0.0731
Epoch 6/10
211/211 ----- 2s 8ms/step - accuracy: 0.9919 - loss: 0.0245 - val_accuracy: 0.9822 -
val_loss: 0.0622
Epoch 7/10
211/211 ----- 2s 10ms/step - accuracy: 0.9940 - loss: 0.0170 - val_accuracy: 0.9822 -
val_loss: 0.0716
Epoch 8/10
211/211 ----- 2s 9ms/step - accuracy: 0.9952 - loss: 0.0143 - val_accuracy: 0.9802 -
val_loss: 0.0810
Epoch 9/10
211/211 ----- 2s 7ms/step - accuracy: 0.9962 - loss: 0.0117 - val_accuracy: 0.9790 -
val_loss: 0.0971
Epoch 10/10
211/211 ----- 2s 9ms/step - accuracy: 0.9954 - loss: 0.0139 - val_accuracy: 0.9775 -
val_loss: 0.0967
```



```
val_loss: 0.0797  
313/313  
Test Accuracy: 97.80%
```

Results

The Fully Connected Neural Network (ANN) was implemented, trained, and evaluated using an in-built dataset. It resulted in a test accuracy of 97.80%.

Learning Outcomes

- Understood how input, hidden, and output dense layers transform features through non-linear activation functions.
- Learned to evaluate scalar prediction error and update weights using gradient descent to minimize loss.
- Gained hands-on experience loading, normalizing, and flattening built-in datasets for fully connected neural networks.